Rapport d'audit de sécurité — FileFlow

Titre	Rapport d'audit de sécurité applicatif (revue de code + scans)
Produit	FileFlow — plateforme de partage de fichiers auto-hébergée
Version	1.1
Date	27 octobre 2026
Statut	Constats H1 à F5 et D1 à D4 corrigés ; points ouverts N1 à N6 (voir § 8)
Propriétaire	Driss OUAAZIZ — Software Architect
Révision auditée	`ef415ba` (branche `master`)
Classification	Confidentiel — contient des scénarios d'exploitation

1. Synthèse

L'audit a porté sur le back-end Rust (`backend/src`, ~8 000 lignes), le front-end React (`frontend/src`), l'image de conteneur et la chaîne de dépendances. Il combine une revue manuelle orientée OWASP Top 10 / ASVS et l'exécution des outils du pipeline (`semgrep`, `trivy`, `gitleaks`, `cargo-audit`).

Les fondations restent solides (CSP stricte, cookies `HttpOnly` / `SameSite=Strict`, révocation JWT par `token_version`, protection des symlinks et du répertoire `.fileflow`, assainissement DOMPurify). En revanche, la revue a mis en évidence **une faiblesse transversale dans le contrôle d'accès aux chemins** et plusieurs escalades de privilèges liées aux fonctions de partage, qu'aucun scanner statique ne détecte.

SÉVÉRITÉ	NOMBRE	CONSTATS
Élevée	2	H1, H2
Moyenne	4	M1, M2, M3, M4
Faible	5	F1 à F5
Information	4	Dépendances (D1 à D4)

Les constats **H1, H2, M3 et M4 ont été reproduits** sur une instance locale jetable (voir § 5). Les autres sont établis par lecture du code, avec le chemin d'exécution cité.

Priorité recommandée : corriger H1 immédiatement (un seul correctif centralisé couvre l'ensemble des handlers), puis H2 et M1, qui concernent la même famille de fonctions de partage.

> **Mise à jour du 27 octobre 2026 (version 1.1).** L'ensemble des constats a été corrigé. Les sections 1 à 7 sont conservées en l'état comme trace de l'audit initial ; le statut de chaque constat, la description des correctifs, un défaut complémentaire découvert pendant la correction de F1 et les nouveaux points ouverts (N1 à N6) figurent au **§ 8**.

2. Périmètre et méthode

- **Revue manuelle** de tous les handlers HTTP : authentification, 2FA, administration, fichiers, téléversements fragmentés, liens publics, demandes de dépôt, partages internes, dossiers partagés, favoris.

- **Vérification dynamique** : binaire de développement lancé sur un répertoire temporaire (port 18080), scénarios rejoués avec `curl`.
- **Outillage** : `semgrep` (règles p/owasp-top-ten, p/rust, p/typescript, p/react, p/secrets), `trivy fs` (vuln, secret, misconfig), `gitleaks` (historique Git complet), `cargo-audit`.
- **Hors périmètre** : tests de charge, analyse DAST complète (voir le document 13), revue de l'infrastructure de déploiement.

---

3. Constats détaillés

H1 — Contournement du contrôle d'accès par segments `..` dans les chemins

Sévérité	Élevée
Catégorie	OWASP A01 — Broken Access Control (CWE-22 / CWE-863)
Emplacement	`backend/src/files.rs:425-479` (`validate_access_inner`), `internal_shares.rs:196`, `collab.rs:176`
Statut	Reproduit

Description. Les décisions d'autorisation comparent le chemin **brut** fourni par le client à des préfixes autorisés (`allowed_paths` des groupes, chemin d'un partage interne ou d'un dossier partagé) par `starts_with`. La résolution vers le disque (`resolve`) intervient **après** et normalise les `..`. Le chemin contrôlé et le chemin réellement accédé diffèrent donc.

Exploitation.

- Utilisateur restreint par son groupe à `/teamA` : `GET /api/files/download?path=/teamA/../teamB/secretdir/payroll.txt` passe le contrôle (préfixe `/teamA/`) puis lit `/teamB/...`.
- Destinataire d'un partage interne **en lecture** sur le dossier `/alice/proj` : `path=/alice/proj/../../<n'importe quoi>` donne accès à toute l'arborescence, sans même détenir la permission `file_read` (le partage interne court-circuite les permissions).
- Avec un partage en **écriture**, la même technique permet d'écrire, renommer, déplacer et supprimer n'importe où (`upload`, `mkdir`, `rename`, `delete`, `move`, `restore_version`, `set_tags`…).
- Les répertoires internes `.fileflow*` restent protégés par `resolve`.

Impact. Rupture complète du cloisonnement entre équipes et des partages ciblés : lecture, voire modification ou destruction, de tous les fichiers du stockage par tout utilisateur restreint ou destinataire d'un partage de dossier.

Recommandation.

1. Introduire une fonction unique `normalize_virtual(path) -> Result<String>` qui **rejette** tout composant `..` (ou normalise strictement), supprime les `.` et les `/` multiples, et produit la forme canonique `/a/b`.
2. Appliquer cette normalisation **en tête** de `validate_access_inner`, et ne transmettre aux handlers que le chemin normalisé (idéalement via un type `VirtualPath` qu'on ne peut construire qu'après validation).
3. Normaliser de même les chemins **stockés** à la création des partages, dossiers partagés, liens publics et demandes de dépôt.

4. Ajouter des tests de non-régression (`/a/../b` , `/a/./b` , `//a` , encodages) et une cible de fuzzing « autorisation ⇔ résolution ».

H2 — Un utilisateur en lecture seule obtient un accès en écriture via une demande de dépôt public

Sévérité	Élevée
Catégorie	OWASP A01 — Broken Access Control (CWE-269)
Emplacement	`backend/src/upload_requests.rs:243-278` et `:321-451`
Statut	Reproduit

Description. `create_upload_request` n'exige que `Permission::FileRead` sur le dossier cible. Le lien public `/api/r/<token>` permet ensuite à **n'importe quel anonyme** de déposer des fichiers dans ce dossier. Un fichier de même nom **remplace le fichier existant** (l'ancien est seulement archivé comme version).

Exploitation (reproduite). Un compte `viewer` (lecture seule, `mkdir` refusé avec 403) crée une demande de dépôt sur `/alice/proj` , puis envoie anonymement `report.txt` : le fichier d'Alice est remplacé par un contenu arbitraire.

Impact. Escalade lecture → écriture, et altération silencieuse de documents par des tiers non authentifiés (substitution d'une facture ou d'un contrat par un document piégé, par exemple).

Recommandation.

- Exiger `FileWrite` via `validate_own_access` (et non `validate_access`) pour créer une demande de dépôt.
- Ne **jamais** écraser un fichier existant depuis un dépôt public : suffixer le nom (`rapport (1).pdf`) ou déposer dans un sous-dossier horodaté.
- Revalider l'accès en écriture du créateur au moment du dépôt (ses droits ont pu être retirés depuis).

M1 — Escalade de privilèges via les dossiers partagés (collab)

Sévérité	Moyenne
Catégorie	OWASP A01 — Broken Access Control
Emplacement	`backend/src/collab.rs:137-222` , `files.rs:453-456`
Statut	Établi par lecture du code

Description.

- `create_shared_folder` n'exige que `FileRead` , via `validate_access` , qui **accepte un partage interne reçu**.
- Le créateur d'un dossier partagé obtient, dans `collab.check_access` , un accès en **écriture** qui **court-circuite les restrictions de chemins** de son groupe.
- `update_members` permet au créateur d'attribuer `can_write: true` à n'importe quel utilisateur, sans vérifier que lui-même dispose de l'écriture.

Conséquences.

- Un éditeur restreint à `/teamA` , qui a reçu un partage interne **en lecture** sur `/teamB/docs` , crée un dossier partagé sur `/teamB/docs` et y obtient l'écriture.
- Un utilisateur en lecture peut accorder l'écriture à des tiers.

- Le contrôle anti-repartage transitif mis en place pour les partages internes (`validate_own_access`) est contourné.

Recommandation. Utiliser `validate_own_access` à la création. Exiger que le créateur détienne lui-même `FileWrite` sur le dossier pour attribuer `can_write` . Ne pas donner d'écriture implicite au créateur au-delà de ses droits propres. Normaliser le chemin stocké (cf. H1).

M2 — Remplacement du second facteur sans réauthentification

Sévérité	Moyenne
Catégorie	OWASP A07 — Identification and Authentication Failures
Emplacement	`backend/src/two_factor.rs:341-368` , `users.rs:413-424` , `two_factor.rs:608-643`
Statut	Établi par lecture du code

Description. `POST /api/auth/2fa/setup` écrase `totp_secret` **même lorsque la 2FA est déjà active** (`totp_enabled` reste à `true`), sans demander ni mot de passe ni code TOTP courant. Une session volée (cookie dérobé, poste déverrouillé) suffit donc à substituer le secret TOTP de la victime par celui de l'attaquant. La victime perd l'accès (hors codes de secours) et l'attaquant contrôle le second facteur. Par ailleurs, `disable` n'exige que le mot de passe, sans code TOTP.

Recommandation.

- Refuser `setup` lorsque `totp_enabled` est vrai.
- Stocker le secret en cours d'enrôlement dans un champ distinct (`pending_totp_secret`), promu seulement par `confirm` .
- Exiger mot de passe **et** code TOTP (ou code de secours) pour `disable` et `regenerate` .
- Incrémenter `token_version` lors de l'activation et de la désactivation.

M3 — La déconnexion échoue (HTTP 500) et ne supprime pas le cookie de session

Sévérité	Moyenne
Catégorie	OWASP A07 — Session Management (CWE-613)
Emplacement	`backend/src/lib.rs:196-208` (handler), `lib.rs:605` (route)
Statut	Reproduit (`POST /api/auth/logout` → `500 Internal Server Error`)

Description. La route `/auth/logout` est déclarée dans le routeur public, hors du middleware `require_auth` . Or le handler extrait `Extension<Claims>` , qui n'est jamais injecté : l'extracteur échoue et axum renvoie 500 **avant** que l'en-tête `Set-Cookie` d'effacement ne soit produit. Le cookie `HttpOnly` reste valide jusqu'à 7 jours dans le navigateur. Sur un poste partagé, l'utilisateur « déconnecté » reste en réalité authentifié.

Recommandation. Rendre le handler indépendant des claims : lire le cookie de façon optionnelle, journaliser si un jeton valide est présent, et **toujours** renvoyer l'effacement des cookies `fileflow_token` et `fileflow_2fa_pending` . Ajouter un test d'intégration sur l'en-tête `Set-Cookie` .

M4 — Absence de limitation de tentatives sur les liens publics protégés par mot de passe

Sévérité	Moyenne
Catégorie	OWASP A07 / A04 — Brute force, consommation de ressources (CWE-307, CWE-400)
Emplacement	`backend/src/shares.rs:376-409` , `upload_requests.rs:321-411`

Statut	Reproduit (12 tentatives erronées consécutives, aucune limitation)

Description.

- `POST /api/s/<token>` et `POST /api/r/<token>` vérifient un mot de passe bcrypt sans aucun limiteur. Les mots de passe de partage sont souvent courts (codes PIN).
- Chaque tentative coûte une vérification bcrypt (~250 ms CPU) : vecteur de saturation CPU non authentifié.
- Le dépôt public **écrit les fichiers sur disque avant** de vérifier le mot de passe (jusqu'à `MAX_UPLOAD_MB`, soit 50 000 Mo dans `docker-compose.yml`).

Recommandation.

- Réutiliser `ratelimit::Limiter` avec une clé `token` et une clé `IP` (verrouillage après 5 échecs).
- Pour les dépôts, exiger le mot de passe **avant** le flux de fichiers (champ multipart en premier, ou en-tête dédié), et rejeter les champs fichiers reçus avant validation.
- Plafonner `MAX_UPLOAD_MB` pour les dépôts anonymes, indépendamment de la limite authentifiée.

F1 — Condition de concurrence sur les liens à téléchargement limité

`shares.rs:359-372` : le contrôle `limit_reached()` et l'incrément `increment_download()` ne sont pas atomiques. Des requêtes parallèles permettent de télécharger plusieurs fois un lien « usage unique ». **Correctif :** réserver le téléchargement (vérification + incrément) sous le même verrou d'écriture avant de servir le fichier.

F2 — Réutilisation d'un JWT après suppression puis recréation d'un compte homonyme

`users.rs:149-207` : un compte recréé repart avec `token_version = 0`, si bien qu'un jeton émis pour l'ancien compte du même nom (`tv = 0`) redevient valide pour le nouveau. **Correctif :** initialiser `token_version` aléatoirement, ou inclure un identifiant immuable (UUID, `created_at`) dans les claims et le vérifier.

F3 — Déni de service par verrouillage de compte

`lib.rs:109-131` : cinq échecs sur un nom d'utilisateur verrouillent ce compte 15 minutes, quelle que soit l'IP. Un tiers peut donc bloquer durablement un compte connu (administrateur, par exemple). Ce compromis est documenté (ADR 0006). **Piste :** verrouillage progressif, ou verrouillage par couple (utilisateur, IP) complété d'un plafond global par utilisateur plus élevé.

F4 — Rétrogradation possible du dernier administrateur

`admin.rs:29-44` / `users.rs:499` : la suppression du dernier administrateur est bloquée, mais pas sa rétrogradation ni sa désactivation via `PUT /admin/users/:u`. **Correctif :** appliquer la même garde (`admin_count() <= 1`) aux changements de rôle et de statut.

F5 — Liens publics non rattachés au cycle de vie du fichier

`files.rs:1281-1325` : `rename` et `move` re-pointent partages internes, dossiers partagés et favoris, mais **pas les liens publics** (`ShareLink.file_path`). Un lien public peut alors exposer un **nouveau** fichier créé ultérieurement au même chemin. De plus, un lien survit au retrait des droits de son créateur. **Correctif :** rebaser ou purger les liens lors de `rename`, `move` et `delete`, et revalider l'accès du créateur à chaque téléchargement.

---

4. Résultats des outils automatisés

OUTIL	RÉSULTAT	COMMENTAIRE
`semgrep` (5 jeux de règles)	0 constat	1 erreur d'analyse sur `documentation/08-api/openapi.yaml:395` (fichier non applicatif)
`gitleaks` (11 commits)	0 secret	—
`trivy fs` secrets / misconfig	0	—
`trivy fs` vulnérabilités	3	DOMPurify, voir D1
`cargo-audit`	4 vulnérabilités, 1 avertissement	Voir D2 à D4

Dépendances

RÉF.	COMPOSANT	AVIS	ATTEIGNABILITÉ	ACTION
D1	`dompurify` 3.4.16	GHSA-cg2g-xh64-v4q7 (critique), GHSA-mrv2-3pqr-j5jr (élevée), GHSA-c6gr-qvv4-5v3f (faible)	**Non atteignable** : concerne `SAFE_FOR_TEMPLATES`, `IN_PLACE` et le mode XHTML, non utilisés par `src/sanitize.ts`	Monter en 3.4.17 (sinon le contrôle CI `trivy` échoue)
D2	`zip` 0.6.6	RUSTSEC-2026-0367 / 0368 / 0369	**Non atteignable** : failles d'**extraction** ; FileFlow ne fait qu'écrire des archives	Migrer vers `zip` 2.x
D3	`crypto-bigint` 0.5.5 (via `rsa`)	RUSTSEC-2026-0387 (timing)	**Non atteignable** : RS256 inutilisé (HS256 seul, ADR 0003)	Ajouter à `backend/.cargo/audit.toml` avec justification, comme RUSTSEC-2023-0071
D4	`tokio` 1.52.1	RUSTSEC-2026-0386 (unsound, `AsyncFd`)	**Non atteignable** : `AsyncFd` non utilisé	Mettre à jour `tokio` au prochain cycle

5. Preuves de reproduction

Les scénarios ont été exécutés sur une instance locale jetable (données de test générées, port 18080). Comptes : `bob` (éditeur restreint à `/teamA`), `carol` (viewer), `alice` (éditrice).

```
== 1. Group allowed_paths bypass (bob restricted to /teamA)
  direct  /teamB/secretdir/payroll.txt -> 403
  via ..  /teamA/../teamB/...          -> TEAM-B CONFIDENTIAL
== 2. Internal directory share escape (alice shares /alice/proj read-only)
  bob reads /alice/proj/../../teamB/... -> TEAM-B CONFIDENTIAL
== 3. Viewer (carol, read-only) creates public upload link
  anonymous upload -> 200
  /alice/proj/report.txt now: TAMPERED
```

```
  carol direct write attempt (mkdir) -> 403
== 4. Logout endpoint
HTTP/1.1 500 Internal Server Error
== 5. Share password brute force: no throttle
200 200 200 200 200 200 200 200 200 200 200 200
```

---

6. Points forts constatés

- En-têtes de sécurité complets : CSP sans `unsafe-inline` pour les scripts, `frame-ancestors`, COOP, Permissions-Policy, HSTS conditionnel.
- Cookies `HttpOnly`, `SameSite=Strict`, `Secure` par défaut ; révocation des jetons par `token_version` (changement de mot de passe, déconnexion globale, modification des droits).
- `resolve()` : protection contre l'évasion par symlink et blocage de `.fileflow*` à toute profondeur, couverts par des tests et du fuzzing.
- Contenus actifs (HTML/SVG/XML) servis en `text/plain` en prévisualisation ; liens publics toujours servis en `attachment`.
- bcrypt avec opération leurre contre l'énumération de comptes ; limitation des tentatives pour la connexion et la 2FA.
- Protection contre l'injection de formules dans l'export CSV d'audit.
- Image distroless statique, utilisateur non-root, 0 CVE système.

---

7. Plan de remédiation

PRIORITÉ	CONSTAT	EFFORT ESTIMÉ	ÉCHÉANCE PROPOSÉE
P0	H1 — normalisation centralisée des chemins	0,5 à 1 j	Immédiate
P0	H2 — droits requis et non-écrasement des dépôts publics	0,5 j	Immédiate
P1	M1 — droits des dossiers partagés	0,5 j	Sprint en cours
P1	M3 — déconnexion	0,25 j	Sprint en cours
P1	M2 — cycle de vie de la 2FA	0,5 j	Sprint en cours
P1	M4 — limitation des tentatives sur les liens publics	0,5 j	Sprint en cours
P2	F1 à F5	1 j au total	Sprint suivant
P2	D1 à D4 — mises à jour de dépendances	0,25 j	Sprint suivant

Contrôles à ajouter au pipeline : des tests d'intégration HTTP couvrant les scénarios du § 5 (pour qu'ils échouent tant que les failles existent), ainsi qu'une cible de fuzzing comparant le chemin autorisé et le chemin résolu.

---

8. Suivi de la remédiation (mise à jour du 27 octobre 2026)

Cette section complète le rapport initial sans le modifier. Les correctifs déjà intégrés sont identifiés par leur commit ; ceux du 27 octobre 2026 sont présents dans l'arbre de travail et seront référencés lors de leur intégration.

8.1 Statut des constats

CONSTAT	STATUT	RÉFÉRENCE	NATURE DU CORRECTIF
H1	Corrigé	`de7a95e`	Normalisation canonique des chemins virtuels (`files::normalize_virtual` : rejet de tout segment `..` en 400, suppression des `.` et des `/` multiples) appliquée avant toute décision d'autorisation ; comparaison par segments (`files::path_covers`) au lieu de `starts_with` ; `allowed_paths` des groupes validés et canonicalisés à l'écriture ; chemins des partages et liens stockés sous forme canonique.
H2	Corrigé	`945dc6c`	Créer une demande de dépôt exige l'écriture **propre** (`validate_own_access(FileWrite)`) ; un dépôt n'écrase jamais (nom numéroté `report (1).txt` , réservation atomique par `create_new` puis renommage) ; les droits d'écriture du créateur sont revérifiés à chaque dépôt (404 sinon).
M1	Corrigé	`d3c6f42`	Autorité dérivée : un membre de dossier partagé n'obtient jamais plus que les droits propres du créateur (`files::own_grant`), vérifiés à chaque accès ; créer un dossier partagé exige `own_grant(FileRead)` , accorder l'écriture exige `own_grant(FileWrite)` .
M2	Corrigé	`ec0d17c`	`setup` et `confirm` refusés (400) si la 2FA est active ; `disable` et `regenerate` exigent mot de passe **et** code (TOTP ou code de récupération), échecs comptés dans le verrouillage 2FA ; `confirm` et `disable` font tourner la session (`token_version` incrémenté, nouveau cookie).
M3	Corrigé	`c920a94`	`POST /api/auth/logout` ne dépend plus des claims : réponse 200 et effacement systématique de `fileflow_token` et `fileflow_2fa_pending` .
M4	Corrigé	`c920a94`	Limiteur `link_limiter` sur `/s/:token` et `/r/:token` , clés `link:<token>` et `ip:<ip>` , 5 échecs → 15 min, réponse 429 ; dépôt public : mot de passe exigé avant les fichiers, plafond dédié `PUBLIC_UPLOAD_MAX_MB` (défaut 100 Mo, borné par `MAX_UPLOAD_MB`).
F1	Corrigé (27 octobre 2026)	—	Voir § 8.2.
F2	Corrigé (27 octobre 2026)	—	Voir § 8.2.
F3	Corrigé (27 octobre 2026)	—	Voir § 8.2.
F4	Corrigé (27 octobre 2026)	—	Voir § 8.2.
F5	Corrigé (27 octobre 2026)	—	Voir § 8.2.
D1	Corrigé	`759429f`	`dompurify` 3.4.17.

CONSTAT	STATUT	RÉFÉRENCE	NATURE DU CORRECTIF
D2	Corrigé	`759429f`	`zip` 9 sans les formats zstd/bzip2 (`default-features = false`).
D3	Accepté et documenté	`759429f`	RUSTSEC-2026-0387 ajouté aux exceptions justifiées (`backend/.cargo/audit.toml` , `backend/deny.toml`) : RS256 inutilisé, HS256 seul.
D4	Corrigé	Mises à jour de dépendances	`tokio` 1.54.1.

8.2 Correctifs des constats de sévérité faible

- **F1 — plafond de téléchargements.** La vérification du plafond et l'incrément sont désormais une seule opération atomique (`ShareStore::try_reserve_download` , sous le même verrou d'écriture) ; si l'envoi du fichier échoue, le téléchargement réservé est restitué (`release_download`). Un test lance 50 réservations concurrentes sur un lien à usage unique et vérifie qu'une seule aboutit.
- **F2 — réutilisation de JWT.** `token_version` est initialisé à une valeur aléatoire (`u64`) à la création d'un compte : un jeton émis pour un compte supprimé ne redevient pas valide pour un compte recréé sous le même nom.
- **F3 — verrouillage de compte.** Le modèle de verrouillage de la connexion est remplacé (il se substitue à celui décrit dans l'ADR 0006) :
  - par IP : 5 échecs → 15 minutes (inchangé) ;
  - par couple (utilisateur, IP) : 5 échecs → 15 minutes ;
  - par compte, toutes IP confondues : 50 échecs (`ACCOUNT_MAX_ATTEMPTS`) → 15 minutes ; ce compteur n'est pas remis à zéro par une connexion réussie et expire seul.

  Un tiers ne peut donc plus bloquer un compte avec cinq mauvais mots de passe : il lui faut une cinquantaine d'échecs répartis sur au moins dix adresses IP. Les compteurs restent en mémoire (perdus au redémarrage) et sont purgés périodiquement.
- **F4 — dernier administrateur.** Il est impossible de rétrograder, désactiver ou supprimer le dernier administrateur **actif** (un administrateur désactivé ne compte pas). La garde est évaluée sous le même verrou que la modification (`UserStore::apply_update` et `UserStore::delete`), ce qui exclut deux opérations concurrentes passant chacune le contrôle. Réponse 400.
- **F5 — cycle de vie des liens.** Les liens publics (`/s/:token`) et les demandes de dépôt (`/r/:token`) suivent la ressource lors d'un renommage ou d'un déplacement, et sont supprimés avec elle, comme les partages internes, dossiers partagés, favoris et métadonnées. À chaque téléchargement par lien public, le serveur revérifie que le créateur est toujours actif et dispose toujours de l'accès en lecture ; à défaut, il répond par un 404 uniforme qui ne révèle pas la cause.

8.3 Défaut complémentaire découvert pendant la correction de F1

Les requêtes partielles (en-tête `Range` , réponse 206) n'étaient pas décomptées, afin de ne pas compter plusieurs fois un même téléchargement repris ou parcouru. Conséquence : un lien à téléchargement limité, y compris « usage unique », pouvait être récupéré **sans limite** par morceaux successifs, indépendamment de toute concurrence.

Correctif (27 octobre 2026). Les liens dotés d'un plafond ignorent l'en-tête `Range` et servent toujours le fichier entier, compté ; les liens sans plafond conservent la prise en charge de `Range` (206).

8.4 Vérification

- Tests automatisés : 103 tests back-end et 114 tests front-end, dont des tests de non-régression dédiés (réservation concurrente, restitution, suivi des liens au renommage et purge à la suppression, seuils de verrouillage, garde du dernier administrateur, `token_version` d'un compte recréé).
- Les correctifs F1 à F5 ont en outre fait l'objet de vérifications de non-régression sur une instance locale en fonctionnement.

8.5 Points ouverts

RÉF.	CONSTAT	SÉVÉRITÉ ESTIMÉE	ÉTAT
N1	Un tiers peut verrouiller un lien public protégé pour tous ses destinataires en accumulant 5 échecs (clé `link:<token>` du limiteur).	Faible	Décision produit en attente (compromis entre protection contre la force brute et disponibilité du lien).
N2	L'image de build front-end repose sur `node:20-slim` ; Node 20 n'est plus maintenu depuis avril 2026.	Faible	À planifier (migration vers une version LTS maintenue).
N3	Les images Docker de base ne sont pas épinglées par empreinte (`@sha256:`).	Faible	À planifier.
N4	La chaîne d'outils Rust n'est pas figée en CI : une nouvelle version de clippy peut faire échouer le pipeline sans changement de code.	Information	À planifier (`rust-toolchain.toml`).
N5	`POST /api/shares` (création d'un lien public) n'exige que `validate_access(FileRead)` : le destinataire d'un partage interne peut publier un lien public vers la ressource reçue.	Moyenne	Décision produit en attente (alignement possible sur `validate_own_access`).
N6	Un partage interne n'est vérifié contre les droits de son émetteur qu'à la création : il reste effectif après la rétrogradation de l'émetteur, jusqu'à sa révocation ou la suppression de la ressource (contrairement aux dossiers partagés et aux liens publics, revérifiés à chaque accès).	Faible	Revérifier `own_grant` de l'émetteur à chaque accès, comme pour les dossiers partagés (voir ADR 0008).